

Assessment: University Library Management System (ULMS)

Course: Database Management Systems (MySQL)

Duration: 2 Hours

Total Marks: 100

Scenario

You are hired as a Junior Database Developer for the "University of Knowledge." The university is modernizing its Library Management System. Currently, they store all data in a single, massive spreadsheet. This spreadsheet is causing data redundancy, update anomalies, and slow queries.

The management has provided you with the current "master table" structure (which is **not** normalized) and a list of queries they frequently need.

Your Task: You must redesign the database (Normalize it), write the SQL DDL to create the tables, and write the SQL queries required by the management.

Part A: Database Normalization (Theory & Design) - [30 Marks]

The current "Master Table" structure is as follows:

```
Loan_Records (  
  Loan_ID, Student_ID, Student_Name, Student_Dept, Book_ISBN, Book_Title, Book_Author, Publisher_Name, Publisher_Address, Date_of_Issue, Date_of_Return, Fine_Amount  
)
```

Assume a student can borrow multiple books, and a book can be borrowed multiple times by different students.

Questions:

1. Identify Anomalies (6 Marks):

- Explain, with examples from the `Loan_Records` table, what **Update Anomalies** and **Insertion Anomalies** are.

2. Normalization Process (18 Marks):

- **a. 1NF (4 Marks):** What is the primary key of this table? Is it in 1NF? If not, how would you make it 1NF? (Assume a student can borrow a book only once on a given date).
- **b. 2NF & 3NF (14 Marks):**
 - Decompose the `Loan_Records` table into a set of tables that satisfy **3NF (Third Normal Form)**.
 - Write the **Table Names** and their **Attributes (Columns)**.
 - Underline the **Primary Key** for each table and mark the **Foreign Keys** with an asterisk (*).
 - Justify why your design is in 3NF (explain the partial and transitive dependencies you removed).

3. Entity Relationship (6 Marks):

- Draw a simple **ER Diagram** or write the **Relational Schema** (e.g., STUDENT(**Student_ID**, Name, Dept)) for the tables you designed in Q2(b). Clearly indicate the relationships (One-to-Many, Many-to-Many) between the tables.

Part B: SQL Implementation (DDL & DML) - [30 Marks]

Based on your normalized design from Part A (you can use your own table/column naming conventions), write the SQL code for the following:

4. DDL - Create Tables (10 Marks):

- Write the `CREATE TABLE` statements for all the tables you designed.
- Ensure you define appropriate **Data Types** (e.g., `VARCHAR`, `DATE`, `INT`), **Primary Keys**, **Foreign Keys**, and **NOT NULL** constraints.
- *Hint: For the "Loan" table, think about what the composite key might be to enforce 1NF.*

5. **DML - Insert Data (4 Marks):**

- Insert at least **3 records** into your `STUDENT` table and **4 records** into your `BOOK` table (or equivalent entities).
- Insert **5 loan records** into your `LOANS` (or `TRANSACTIONS`) table. Ensure that at least one student has borrowed more than one book.

6. **SQL Query - Joins (8 Marks):**

- Write a SQL query to display the following details for all students who have currently issued books (i.e., where `Date_of_Return` is NULL):
 - `Student_ID`, `Student_Name`, `Book_Title`, and `Date_of_Issue`.
- **Constraint:** You must use an **INNER JOIN** to combine the necessary tables.

7. **SQL Query - Aggregation & Group By (8 Marks):**

- Write a SQL query to find the total number of books issued by each department.
 - The output should show: `Student_Dept` and `Total_Books_Issued`.
 - Sort the results by `Total_Books_Issued` in descending order (highest to lowest).
-

Part C: Advanced SQL (Nested Queries & Joins) - [40 Marks]

This section tests your ability to solve complex problems using the skills you have learned.

8. **Nested Query (Subquery) (12 Marks):**

- **Problem:** The management wants to give a "Bookworm Award" to the student who has borrowed the **most** number of books overall.
- Write a SQL query to find the `Student_Name` and `Student_ID` of the student(s) who have the maximum count of loans.
- **Constraint:** You must use a **Nested Subquery** (and cannot use `LIMIT` or `ORDER BY ... DESC LIMIT 1`).

9. **Correlated Subquery (12 Marks):**

- **Problem:** The library wants to identify students who have borrowed books from the same department they belong to (e.g., A Computer Science student borrowing a Computer Science book). Assume the `Book` table has a `Book_Subject` or `Dept` column that matches the `Student_Dept`.
- Write a SQL query to list all distinct students who have borrowed at least one book related to their own department.
- **Constraint:** You must use a **Correlated Subquery** with `EXISTS`.

10. Complex Join (Multiple Tables) (16 Marks):

- **Problem:** The library wants to generate a report for overdue books. A book is overdue if the `Date_of_Return` is past `CURRENT_DATE` and `Date_of_Return` is NOT NULL.
 - Write a SQL query to generate the following report:
 - `Student_Name`, `Student_Dept`, `Book_Title`, `Publisher_Name`, `Date_of_Issue`, `Date_of_Return`.
 - Additionally, calculate a column called `Days_Overdue`. (Use `DATEDIFF(CURRENT_DATE, Date_of_Return)`).
 - **Constraint:** You must join at least **4 tables** (Student, Loan, Book, Publisher) using **INNER JOIN** syntax.
 - **Filter:** Only display records where the book is overdue.
-

Bonus Question (Optional - Extra 5 Marks)

11. Subquery with ANY/ALL:

- Find the `Book_Title` of books that have been issued more times than the *average* number of times all books have been issued.
- *Hint: Use a subquery with `HAVING` or `> ALL`.*